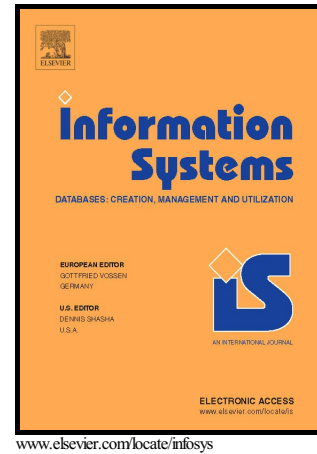


Author's Accepted Manuscript

The Performance Metric for Enterprise Service Bus (ESB) in SOA System: theoretical underpinnings and empirical illustrations for information processing

Robin Singh Bhadoria, Narendra S Chaudhari,
Geetam Singh Tomar



PII: S0306-4379(15)30195-2
DOI: <http://dx.doi.org/10.1016/j.is.2016.12.005>
Reference: IS1193

To appear in: *Information Systems*

Received date: 4 December 2015
Revised date: 10 November 2016
Accepted date: 27 December 2016

Cite this article as: Robin Singh Bhadoria, Narendra S Chaudhari and Geetam Singh Tomar, The Performance Metric for Enterprise Service Bus (ESB) in SOA System: theoretical underpinnings and empirical illustrations for information processing, *Information Systems*, <http://dx.doi.org/10.1016/j.is.2016.12.005>

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting galley proof before it is published in its final citable form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

The Performance Metric for Enterprise Service Bus (ESB) in SOA System: theoretical underpinnings and empirical illustrations for information processing

Robin Singh Bhadoria^a, Narendra S Chaudhari^b, Geetam Singh Tomar^c

^aDiscipline of Computer Science & Engineering, Indian Institute of Technology Indore

^bDept. of Computer Science & Engineering, Visvesvaraya National Institute of Technology Nagpur, Maharashtra 440010, India

^cMachine Intelligence Research Labs, Gwalior 474011, India

robin19@ieee.org

nsc0183@gmail.com

gstomar@ieee.org

Abstract

Now days, the businesses are going online and e-Commerce industry is on its boom. In this changing era of development, services are to be Robust, Agile, Accessible and Available to its clients. For secured and guaranteed delivery of services, every big organization is shifting their service delivery model to Enterprise Service Bus (ESB). It promises to set up a strong guideline to build System Oriented Architecture (SOA) system, which leverages multiple services from different application domains. This paper presents an analytical survey of ESB on different parameters influencing the performance of SOA in the present changing scenario and service patterns.

Keywords: Enterprise Service Bus, Service-Oriented Architecture (SOA) System, Service stack, Service patterns, Message Routing, Enterprise Integration Pattern.

1. Introduction

In era of rapidly changing technology, there is serious demand for service integration of different application domains sharing of various IT assets within as well as outside organization. A consequence of this change in demand is highlighted in [3, 22] as follows:

- If an organization expands its business, then it requires restriction free environment for its business.
- Changing demand of market needs elastic, scalable and compliant system architecture.
- Complete utilization of existing IT assets.
- Compliance with maintenance, reconfigurable and reusable with system services.

In the context of issues highlighted above, ESB is an appropriate solution. It is a layered architecture for service integration, reusability, and compatibility. Due to the distributed behavior of ESB, it has no region boundaries i.e. services can be accessed from anywhere. ESB also supports multiple data model and transform information to increase agility and accessibility of SOA systems [29]. Furthermore, it can dynamically handle simultaneous requests from multiple interfaces (end-point). Routing is core feature of ESB, which provides complete assurance of message delivery between service client and provider. ESB also supports various aspects of SOA including resource sharing, message handling, translation, data transformation, protocol conversion, service versioning, and extending several others features of SOA [2, 5].

The word “Enterprise Service Bus” was first introduced by Roy W. Schulte from Gartner Group in 2002. In June 2004, David Chappell wrote a book -*The Enterprise Service Bus*, which briefs about several aspects of service bus. In 1987 *Teknekron Corporation, USA* project with Goldman Sachs first introduced “*The Information Bus (TIB)*”, which defined integration and delivery of market information like stock quotes, news and other financial figures. In 1997, Vivek Ranadivé founded a company named TIBCO (stands for *The Information Bus COmpany*), which provided service, especially, other than financial sector and released TIBCO Active Enterprise suite. But there is a great demand in industry as well as business for acceptance of ESB with SOA system as new challenges [16, 35].

This paper focuses on issues, which are important in deciding flexibility, reliability and performance metric for ESB. It also describes core design principles, centric view of ESB in SOA system with service stack and its logical

working. It also covers the detailed routing aspects for ESB in supporting service between service requesters and providers [32].

The rest of the paper is organized as follows: Section II explains about architecture framework for ESB with its core principle for designing. Further, it highlights the importance of service stack with regard to delivering services in SOA system. It also presents a comprehensive architecture view for ESB and what ESB actually comprises. Section III illustrates the concept of message routing and mechanisms associated with message transformation. It also presents a brief discussion on the classification of routing techniques. Section IV describes the detailed analysis and comparison of exiting ESB product in the market for companies like IBM, Microsoft, Fiorano, Oracle service bus, WSO2, Sonic ESB and etc. Section V gives the insight view for gaps and threats associated with the use of ESB. Section VI concludes the paper.

2. Design and Architectural Framework for ESB

ESB is a software framework that manages service and its integration with other services on a common platform. It also provides necessary infrastructure support to implement message routing, protocol translation and message transformation. It also federates services from legacy application domain and provide loose coupling between them. With ESB, one can design develop, deploy, and monitor services at runtime. This improves the concept of re-usability in ESB [53].

2.1 Essential Design principles for ESB

Recent years have noticed a fast change in the market for Enterprise Service Bus (ESB) technology and its associated integration software. Business organizations are choosing mediation support for their architecture enhancement [4]. ESB as a mediator agent supports in interconnectivity for multiple services i.e. *Service Orchestration*. Such communication is usually based on exchanging messages between the two parties. It allows ESB to receive messages from clients, process them accordingly and forward it to designated service point (usually end-point in ESB). ESB provides multiple interfaces or end-point for different channels to support services. It also incorporates several types of IT resources [2, 17]. The relationship between service requesters, providers and ESB is depicted below in Figure 1.

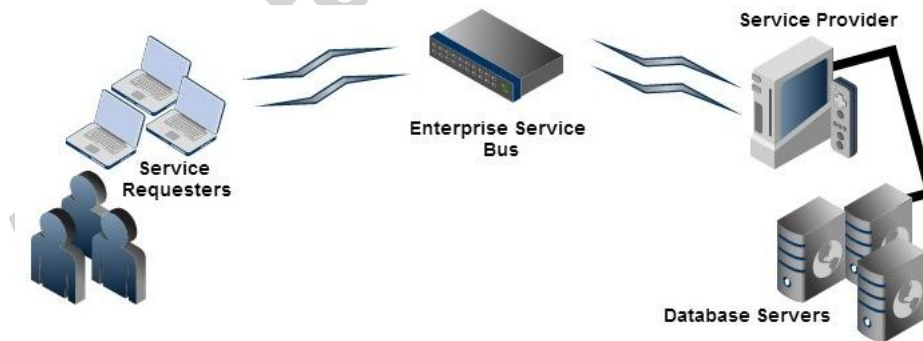


Figure 1: ESB Infrastructural Concept

The support for a variety of mediation allows ESB to fulfill two core principles:

- (1) Service virtualization
- (2) Aspect-oriented connectivity

- (1) *Service Virtualization* defines the capability of ESB to support virtually – the transmission protocol, interaction pattern, interface for communication and most importantly its identity during complete service interaction [7].
- **Transmission Protocol:** It enables ESB to process incoming request messages and transforms into most compatible format with other service requesters. It sends and receives messages through various transport layer protocols like HTTP(S), SOAP, and JMS etc.
 - **Interaction Pattern:** This allows ESB to support different patterns that help in recognizing message formats, and service integration. The ESB act as a mediator between service provider and its associated client. This feature is also helpful in communication between two or more different ESBs together [41].
 - **Environment Interface:** This permit ESB to incorporate service requesters and providers with different interface in communication. It also known as “*end-point*” in ESB. It also enables multiple interfaces to provide accessibility through different end-point within ESB. For example, service client and provider should not use identical interface in accessing the particular services.
 - **Reconcilable Identities:** During communication, service requesters need not to know its identity (like its address). It is the responsibility of ESB to manage such information (identities) for every service requester. In this way, any service requester can be served by any service provider at any time over the internet.
- (2) *Aspect-oriented Connectivity* is second important point of designing principle in ESB. Connectivity has always been a crucial issue in establishing communication between multiple services in SOA Systems [54]. The problem of unavailability of services may result into Denial of Service (DoS) in some condition and such issue could be avoided by adopting ESB as solution. This also involves issues related to connectivity, logging and auditing of services, service validation & security, service handling & management. Such issues must be kept in mind while designing framework for any ESB. It also builds the overall Quality of Service (QoS) features for ESB [14].

2.2 Centric view of An ESB

For Business demand in modern era of computing, ESB could help in establishing measures for desirable solution that form business agility and robustness. These business services must be governed and monitored for better performance and it could be deliver with aid of well define service stack [56]. The service stack is set of service in SOA System reference architecture that extends the features of it. It is well depicted in Figure 2.

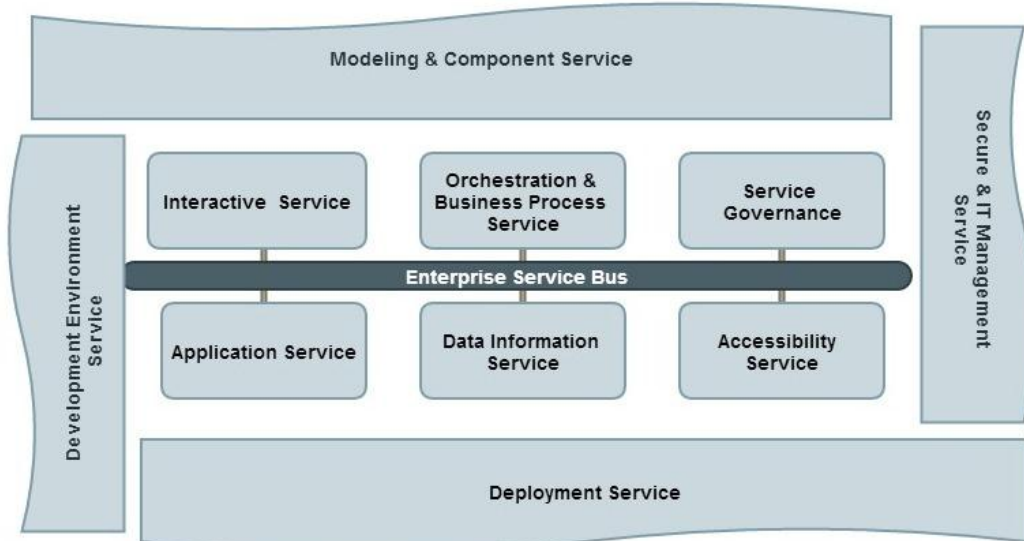


Figure 2: Service Stack in ESB

(a) *Service Stack*

ESB provides several services like– interactive services, Orchestration & Business process service, data information service, application, service, accessibility services and service governance [13]. Such services within the ESB framework are labeled as *services stack*, which help in execution and layout of different forms of services. It also provides compatible and sophisticated development environment to service requesters in achieving on-demand services [20]. These services are as follows:

- *Interactive Services*: Such services provide essential features in delivering IT functionality and data. These services support mediation in communication between multiple services for better accessibility.
- *Orchestration & Business Process Service*: Service orchestration defines as coordination between multiple services and described as a single aggregated unit to facilitate interaction among them. These services also provide the control capabilities to manage workflow and integration.
- *Data Information Service*: Such services federate, replicate and transform information from different data sources. It also provides the necessary supports in message transformation, whenever it is invoked by some service requester.
- *Application Service*: It allows the implementation of core business functional logic that is accessible within the particular business organization. Such services are actually executed and invoked by service requesters and support other service component in service architectural framework.
- *Accessibility Service*: It provides support in making services available to requesters. ESB act as a mediator in allowing such services that are delivered via SOA. It also helps in incorporating service with its associated data. Accessibility to the particular services, governs the security policy associated with its service provider.
- *Service Governance*: It provides a monitoring to the content associated with service. It also helps into costing, planning and configuring for on-demand rental services. It guaranteed the accessibility for particular service and its scope. It provides the guideline for preparing Service Level Agreement.

(b) *Secure & IT Management service*

Security and management services are placed as separate entity with ESB service stack. Since IT related services have solution wise scope and it requires better active coordination and cooperation with other service components. Security of a service is defined as preventing and protecting access to a particular service in specific domain. Restriction on service can be limited i.e. not all requesters have the same permission to access the service. Accessibility to particular services would be preferred on basis of policy associated with service contract. ESB does the authorization in delivering services that can be configured to get an access. For example, web services can be spoofed and can conceal using certain fraudulent methods. To prevent from such stealing, developers use *WS Security* which is an extension to SOAP [32, 33].

(c) *Development Environment service*

These services help in building integration logic and connectivity issue. Equally, such services support ESB into services' discovery, registration and publishing (in regards to *service registry*). With this environment, service can also be model, edit, and test and packed for deployment [16, 31]. Also, it allows modification of service metadata to influence dynamic behavior of a service.

(d) *Deployment Service*

Deployment is process of enabling service application into testing or business environment. It tests the services before publishing it to constraint business environment. Further, it also binds these modules as a

standard application package for deployment [13]. It also resolves issues related to capacity, planning, testing, service hosting, service dependencies and service versioning compatibility.

(e) *Modeling & Component Service*

A Service component is a piece of service which helps in implementing some business logic via interfaces or end-points in ESB. Components may interact or request other components for communication purpose; this is called as *service references*. It federates multiple components associated with particular service to implement loose coupling which expected to be independently deploy of services. These components are plug-in and plug-out with ESB [44]. It also supports assembling of services into group called *modules*. This module actually provides complete integration solution and is well defined in *Service Component Architecture (SCA)*. These modules work independently and do not affect behavior of other modules [27].

Service Modeling is a concept of organizing services to fulfill demand for service requester. It also helps in improving interaction between services. It does support the functionality like routing, transformation, and handling of messages [32]. It provides support for route discovery, validation and transmission priorities. It logs the monitored data for interaction between services. For example, *Service qualifiers* define quality of service that describes a set of communication characteristics needed by a service application.

Table 1: Shows Monitoring Framework, comparative issue and Integration of SOA with ESB

Reference	Monitoring Framework	Comparative Point	Integration with SOA	Comment
Sixto Ortiz Jr. (2007)	Talk about basic building structure for ESB as EAI and its origin in mid-1980's	Does not made any comparison, just presented roadblock for service bus	Show importance of SOA and its integration with standardized web-services like XML for data description; HTTP for message transfer; SOAP for message exchange; WSDL for service description and UDDI protocol for discovering & publishing services	Talk about advantage of ESB over application integration approach
Jiangiang Hu et. al. (2008)	Does not talk about Monitoring framework in ESB	No Comparison	Discuss about "ESBsoa" Methodology, but not unified adapter mechanism	Also restrict availability & Scalability
Deng Bo et. al. (2008)	Explain Event driven architecture for Complex Event Processing	Service selection made on basis of greatest "benefit/Cost" ratio	Discuss ESB Technology with SOA and event driven architecture	Proposed algorithm for event selection, aggregation and Greedy service
Jianwei Yin et al. (2009)	Present "JTangSynergy" which is dependable ESB framework	Design system architecture for Chinese Healthcare Service Integration	Focusing on Service Integration using pluggable platform	This proposed system support service recovery by dynamic service replacement module (DSRM)
Gang Li et. al. (2012)	Discuss Hub Model in EAI for large scale integration	Compare ESB functionality with Soft System Bus, which is main component of Persistent Computing System (PCS)	Present ESB as infrastructure for integrated and flexible end-to-end SOA	Present ESB as Message Passing Engine
Marek Psiuk et. al. (2012)	Introduced ESB Meta Model (EMM) with monitoring framework architecture	Comparison made on message model, topology model and measuring task	Discuss ESB based framework for SOA system	This proposed model gather data related to ESB entities like Message transmission, Topology discovery mechanism, Message flow etc.
Keshi et. al. (2014)	Integrated framework based on Ontology and ESB	Build model for Heterogeneous Systems	Implement SOA based ESB for enterprise knowledge management	Discuss about Ontology Structure of System Integration
Ondřej Hrcuba et. al. (2015)	Presents framework that integrate light weight client with heavy weight ESB	Introduced "REST based ESB gateway (REG)"	Extends the idea of integrating Multi Agent System (MAS) and SOA using RESTful service architecture	Discuss Ontology-based model for semantic representation for information

Table 1 presents different work done by different authors in analyzing ESB. Initially, ESB framework used as Enterprise Application Integration (EAI) and that grows towards the large scale integration as discussed in [15]. Such integration also support with other application domains like Persistent Computing System (PCS), Multi Agent System (MAS), and Heterogeneous Systems etc. This framework could federate light weight client services with heavy weight computing services on common platform. Service orchestration could be possible in such environment

with RESTful gateway [42]. Knowledge management techniques like ontology had been adopted in enterprise solutions in [44].

2.3 What does ESB actually comprises?

ESB promises to build a decentralized architectural framework that supports several services into a unit called service bus. This integration of services defines a platform for serving requests from multiple requesters simultaneously. ESB is not only architectural framework, but also provide highly versatile and distributed behavior [32]. Figure 3 illustrate actual inside conceptualization of ESB.

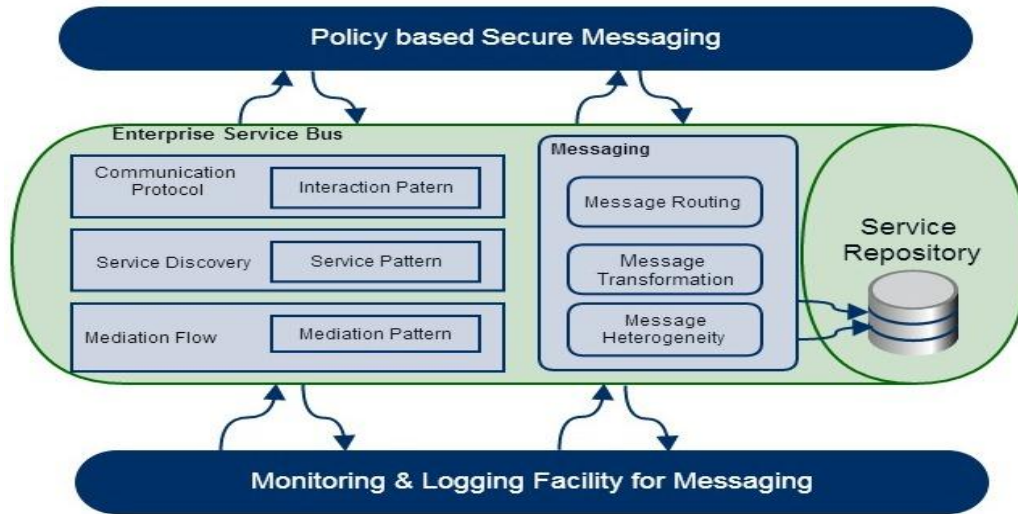


Figure 3: Inside View for ESB

(a) Service Repository

The service Repository (also *service registry*) manages the metadata that store important information related to services at centrally located repository. It is used to encourage reusability and prevent duplication of service. The reuse of service components is a prime contributing factor to reduce cost and time in ESB. It also support in task related to scope of services and policy describing service level agreements [28]. The above Figure 3 depicts the position of service registry.

(b) Policy based Secure Messaging

It supports in message interaction and communication in ESB. If a message from higher or lower layer arrives, then there must be validation check on its access for authentication purpose. It uses digital signature and web service-security based authentication for better and secured messaging in ESB. It also involves authenticating users and access control on resources [17, 31].

(c) Communication protocol

This allows conversion of protocols required by multiple services during communication. ESB itself, in fact, does not provide communication protocols, but it supports multiple communication protocol like HTTP, FTP, REST, SOAP, DCOM, RFC etc. These protocols are used for establishing connection between service requester and provider. It also supports the necessary feature for translation of message format, which is not compatible either to requester or to provider [33, 34].

(d) *Service Discovery*

Service discovery is a part, where implemented/published service can be discovered through service registry. This phase is used by service requester to discover implemented service. It has been observed that services face the problem of service versioning during its discovery. Service Versioning is defining as incompatibility in recognizing service format that may not be identified by the underlying protocol [33]. The reason behind this incompatibility is unidentified message format. For example: consider a situation in which the service provider is using version 2.0 of the particular service and that Service is being requested by any Requester 'A', but Requester 'A' does not recognize the service version. So, there is a problem of Service versioning. In this case, ESB can be used to make transformations from an unidentified version to a known compatible version with the help of service registry.

(e) *Mediation flows*

The word meditation means negotiation that is conducted by some impartial party in resolving differences. To support service availability, connectivity and virtualization - mediation flows play an important role into it. Mediation helps in understanding and providing compatibility for messaging between multiple services. It also monitors the workflow and processes the received messages from requester and then forwarded to particular service provider. Mediation varies its performance from a single interaction to multiple interactions of services in regards to reusability of services. It also uses distinct patterns of message to its processing, also known mediation patterns. ESB uses several ready-made mediation patterns for service discovery [19].

(f) *Messaging in Service Bus*

ESB must support interaction through messaging between multiple services. This could help in improve availability, reliability and flexibility issues in ESB. Messaging has three important aspects related to routing, transformation and Heterogeneity as discussed below [16].

- *Message routing:* Routing provides the shortest path to deliver messages. It supports multiple routing transport protocols like E-mail (POP/SMTP), FTP, JMS, HTTP(S), WS-Reliable Messaging. When a message contains insufficient information of related to data, then it is difficult to route it. At this time, such message must be enriched with some additional information through data available in the service registry.
- *Message Transformation:* It involves the services that aggregate or enrich the XML payload of messages. It also supports XML and Non-XML data transformation. It follows dynamic service selection to transform message payload based on its header. For such transformation facilities, it utilizes XML standards such as Extensible Style-sheet Language Transformations (XSLT). It does support XML based cache for retrieving the previous sent message [55].
- *Message Heterogeneity:* It supports multiple messaging services to maintain its workflow like synchronous & asynchronous, publish & subscribe service etc. It also provide multiple message formats like SOAP (with its attachments), E-mail, XML, JMS headers, Message format Language (MFL). Heterogeneity in message also provides Mix and Match messaging services, WS reliable to unreliable messaging service etc.

(g) *Monitoring & Logging facility for Messaging*

Monitoring is defined as analyzing the situation for service integration at runtime, especially for messaging. A service provider can monitor the content for service and gather all kind of managerial data like auditing log, error reporting or QoS related data [6]. It also prepares statistics for messaging and its payload. It provides real-time observation and dynamic resource loading which includes the logs and alerts that could be scanned and recorded at organizational level.

Table 2 shows the comparative analysis on how different authors have discussed about different perspectives of capacity, planning and experimental simulation in ESB. This table point out the keen issues related with high

performance computing techniques. Testing is very crucial issue in ESB that could be used in phase of capacity planning in [3]. Event driven architecture could also be combined with SOA system via ESB. Relational algebra could improve efficiencies in event processing based system in [26]. Scalability and load handling issues could be performed with direct proxy, content proxy and transformation proxy. This is experimented on three ESB products- Mule, WSO2, and Apache ServiceMix in [19]. Parallel application bus could be introduced with ESB based architecture to improve multiple services processing as discussed in [43].

Table 2: Shows Capacity, Planning and Testing with Existing ESB

Reference	Capacity & Testing	Simulation	High Computing Techniques	Comment
Ken Ueno & Michiaki Tatsubori (2006)	Test on different ESB related to costing, mediation and performance issues	Experiment for web services with no ESB, ESB Without mediation, ESB with mediation	Discuss about Ultra-light service and capacity testing techniques under high load to determine its maximum performance	Good Analytics for Planning & Testing issues
Deng Bo et. al. (2008)	Test and selection that service yield higher benefit per cost ratio	Experiment on service instance for all associative event processing services	Absence of Relational algebra based event stream processing engine in existing ESB	Presented good analysis for complex event processing system
Sanjay P. Ahuja et. al. (2011)	Test scalability & Load handling with Direct Proxy, Content Based Routing Proxy and Transformation Routing Proxy Scenario	Simulated results on ESB like - Mule, WSO2 and Apache ServiceMix	Explain core functionality of ESB w.r.t. virtualization, mediation and content based routing.	Also, presented subjective assessment on parameter like API, Installation, Ease of development, online support for these 3 ESBs
MarekPsiuk et. al. (2012)	Involves testing topologies that support in service containers using JBI different versions	Experiment made on ServiceMix, OpenESB and JSR-208 ESB products	Implemented in Java Business Integration (JBI) that standardized ESB Patterns	Discuss Aspect Oriented Programming (AOP) that support modularity & service orchestration
RidhaBenosman et. al. (2013)	Demonstrate Massively Parallel Application Bus (MPAB), which is ESB oriented architecture	Discuss operation mode of P-Instructions	Shows the functionality of Multiplexing for massively parallel processing of distributed and delay-insensitive application	This could be future solution for complex processing like core banking, Big data simulation etc.

Table 3 illustrates the security aspect of services arises with ESB. Also, justifies the protocol specification that are related with WS-security based authentication, service level agreement (SLA) and encryption. Discuss about error and logging report presented with architecture for SOA [31]. Presented service bus security as essential factor of concern into ESB architecture and calculate error for performance modeling and its validation. This experiment has done on BEA AquaLogic ESB product [10]. ESB could extend the functionality of window communication foundation to offer multiple end-points for published services. Propose the architecture that leverages SOA methodologies in grid computing [7]. Error handling for message with architecture for EAI based ESB is well discussed in [12, 21].

Table 3: Presents Security, Policy, audit and logging facility for ESB

Reference	Security	Driven Policy	Preventions	Comment
Jiang Ji-chen et. al. (2006)	Present Enterprise Security concept using Single-Sign-On (SSO), Lightweight Directory Access Protocol (LDAP) Integration	Discuss idea of digital signature, WS-security based Authentication and encryption in ESB	Establish SLA / SLO based on event and performance	Brief about Architecture for ESB
Yan Liu et. al. (2007)	Provide Service Bus Security as High level ESB Architecture	Discuss Service Level Agreement (SLA) alerts including message security, authorization, authentication	Calculates error of performance modeling and validation	Present case study for Loan Application use Mule, BEA AquaLogic Service Bus (ASB)
Jiangiang Hu et. al. (2008)	Discuss SOAP security framework in business Integration	Discuss Policy and its definitions in service adaptation at service layer	Define security authentication at - Channel Layer, Service Layer of SOA based EAI environment	Discuss transport adaption, service adaption and common services in SOA based ESB
Alaa M. Riad et. al. (2010)	Explained security aspects for SOA based grid computing	HTTP transport and WS-specifications, Proposed SOA based Grid computing	ReliableSession and TransactionFlow protocols	Discuss Windows Communication Foundation (WCF) as a message-based architecture
Jieming Wu and Xiaoli Tao (2010)	Define Security as core functionality of ESB	Use open standards like HTTP(S) to JMS, FTP to a file batch, and SMTP to TCP	Discuss O2 models -Java Authentication and Authorization Service(JAAS) and Acegi Security	Present ESB as validate connective Route Transform

3. Message Routing Functionality in ESB

Enterprise Service Bus (ESB) primarily focuses on message routing, transformation and its handling. It also ensures the necessary infrastructure support for reliable routing of messages between multiple services [11]. Traditional communication via ESB usually follows a rule based mechanism for processing and configuration of messages. This technique does not provide mechanisms for dynamic routing of messages. However, there exist a lot of work in literature that extends this basic functionality through the use of abstract routing path (ARP) and abstract routing table (ART). A good discussion on such approaches is available in [23]. The work done in [24] is further adding method that allowing dynamic reconfigurable routing and extends the functionality to support dynamic routing mechanism. The limitations of ESB could be overcome by allowing dynamic message routing, in which routing path is computed during on runtime.

In literature, there are two broad categories for reliable message routing:

- (1) *Web-based Service Reliability*: It is a SOAP-based protocol for exchanging messages with assured delivery without its duplicity [52]. It uses an independent protocol that allows messages to be exchanged between different services. Further, it provides interoperability in Web services for SOAP binding [1].
- (2) *Routing Pattern*: It defines the message routing pattern in supporting service composition. However, such routing mechanism maintains static routes only and a list of alternate routes has to be provided well in advance. This mechanism also provides the solution in which method is directly applicable for service platforms and integration tools like- TIBCO, IBM WebSphere, ServiceMix, Mule, Sonic ESB and many [41].

The generic functionality for ESB architecture is made up of four parts:

- *Message Mechanism*: It is the functionality of ESB that provides handling of messages between multiple services. It also allows the services to interact with other existing service on a common shared platform.
- *Message Transformation*: It involves the conversion of message to a particular format that can be understood by both communicating service parties. It also provides a layer of abstraction to make the platform technology independent.
- *Routing Mechanism*: This process prevents loss of messages and guaranteed its delivery. It also helps in providing a reliable route to navigate message from its clients to the appropriate service provider.
- *Message Container*: It is the actual entity which carried data and its associated information during communication and services are actually deployed in these message containers. It also hosts the service to receive messages from other containers.

The message functionality of the service bus could help in forwarding and accepting messages from service requesters. Such mechanism has been adopted by work in [23, 25]. Figure 4 pasteurizes the routing mechanism with help of abstract route table (ART) and abstract route path (ARP) along with business logic processing engine.

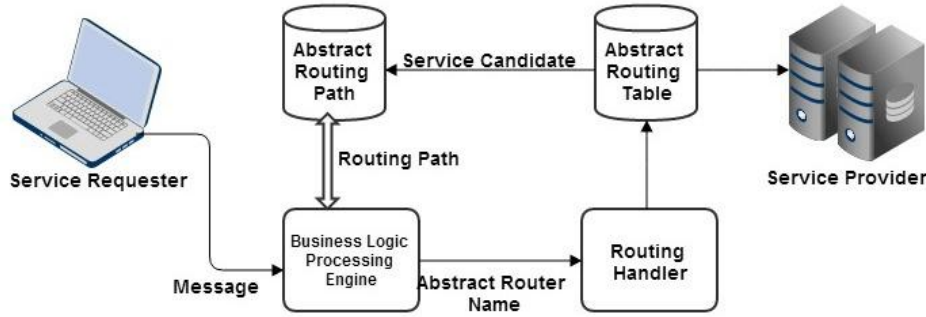


Figure 4: Message Routing in ESB

ARP is used to store paths for routing the messages and also provide to route maps to different abstract routers for service client and its associated provider's URI (Uniform Resource Identifier). Generally, ARP identifies the route with help of ART in terms of *abstract service names*. It is based on the principle of *store and search* for service names and thus, determining the actual service provider. After extracting the actual address, service provider replies with this address to service request rather than just using a service name.

Business logic engine recognizes the potential routing path from ARP. Then, route handler chooses the service provider at run time and checks the content of the routing message. Finally, route handler forwards the message to the corresponding service provider in the route map for further processing [23].

3.1 Classification for Message Routing into ESB

One of the major issues in ESB routing mechanism is dynamic route determination. This requires the evaluation of service that must be done on run-time to check service availability, and reliability. Broadly, classification in message routing in ESB can be categories with following three:

- (1) *Content-Based Routing (CBR)*: It follows message driven routing methodology and widely accepted as an important characteristic in Enterprise Service Bus. In CBR, routing path is identified by considering message content and applying a set of calculations on that content. It usually depends on criteria like message type & its value. For better performance with CBR, a good practice is to enrich the requester's message header with destination address. Figure 5 displays CBR mechanism for communication between two - Service 'A' and Service 'B' [25].

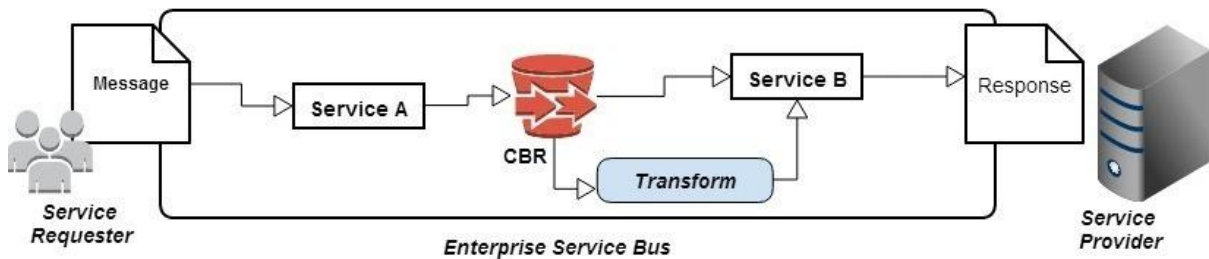


Figure 5: Content-Based Routing mechanism

- (2) *Pattern-Based Routing*: In traditional routing, services are generally retained by containers and service composition is specified via an execution language, e.g. BPEL. Service container is used to define the interoperability between services for cooperation and coordination. However, this methodology has a serious drawback in terms of performance, especially for complex service applications.

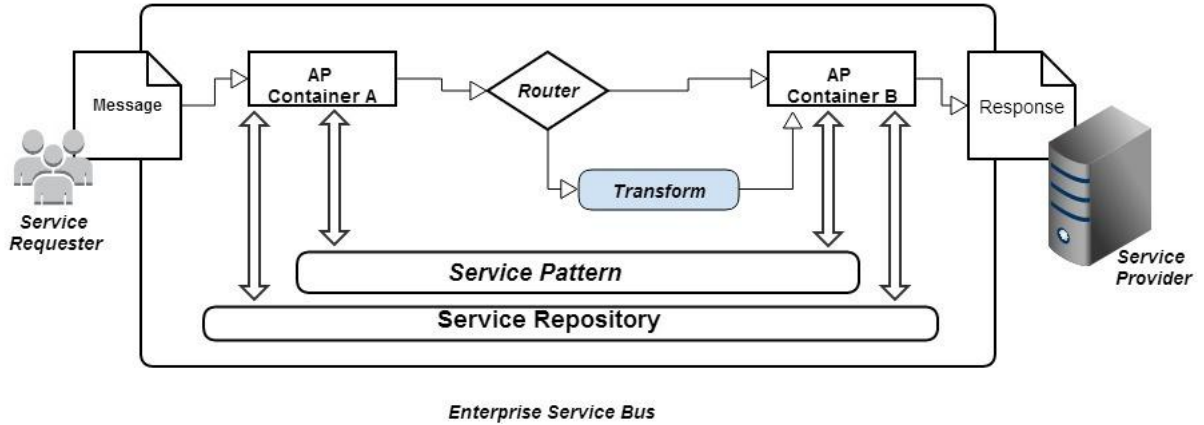


Figure 6: Pattern-Based Routing Mechanism

A possible solution for above problem is Pattern-Based Routing. Figure 6 illustrate basic mechanism for pattern based routing in ESB. It introduces *Application Patterns (AP)* that provides an environment supporting in service orchestration .AP handles the messages from its service requester and then, processes it with the help of existing service patterns that have been provided from its service provider. These Service Patterns provide necessary support for identifying message format and its type by querying the *service pattern repository*. AP container sends this processed message to *Router*, which transforms it to a mode compatible with service provider.

- (3) *Dynamic Routing*: It defines the path for routing that must not be predetermined or fixed. When any client (service requester) demands for any service access, then ESB assigned and allocate a unique task ID to that requester. Now, this task ID is assigned and inserted into the message envelope or also called *service/message container*. This container would be plug-in to ESB and *Routing Manager* navigates this message to appropriate service provider. But path could be modified at runtime [23].

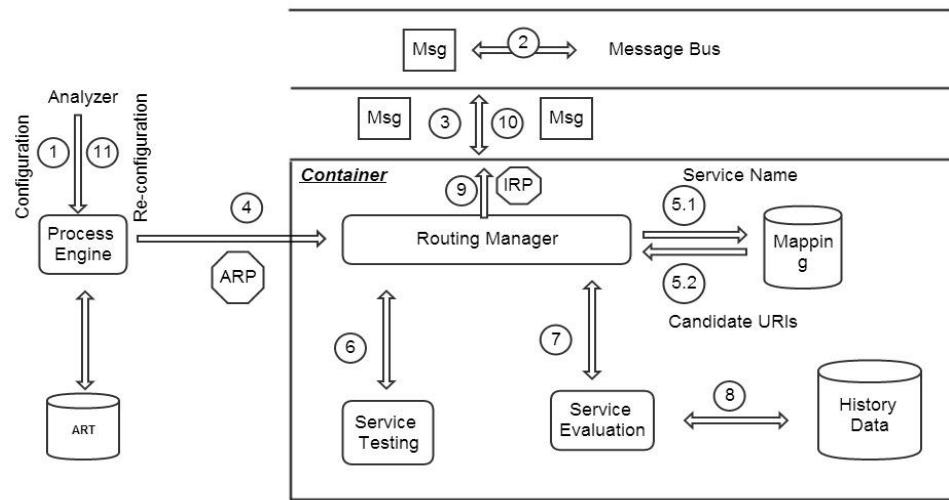


Figure 7: Dynamic Routing Mechanism in ESB

Figure 7 explains dynamic message routing and its management with ESB. A brief description is given below:

- Initially, message request goes to the analyzer component for locating the workflow. The Analyzer further decides the business requirements for this request. Then, the process engine determines the number of possible paths or routes. This routing path is stored into abstract routing table (ART).

2. During execution, messages are exchanged between multiple services. Thus, service bus navigates these messages via assigned and allocated route that defined in ART.
3. Each service is connected via end-point in ESB and these services are hold by service container. During interaction process, messages which have been received through service bus, is converted it to appropriate message format and then forwarded to further processing.
4. In next step, container has to choose service provider at runtime. Therefore, it requests the process engine to obtain new route path from the ART. After that, it extracts service name for further processing.
5. Each container has local mapping table and this table consists of service names. Now, container finds the abstract service name to map its associated candidate service providers (5.1) and return with list of candidate service providers' URI (5.2). This all would be controlled by *Routing Manager*.
6. If there are multiple services for the same functionality, then it uses specific runtime testing on each service.
7. In the evaluation step, when testing is completed, services are analyzed based on certain predefined standards like -its response time etc.
8. Result of testing and its associated evaluation of particular service is kept into historic data.
9. Now, the container replaces the abstract service name in ARP with service provider's URI. It creates the Instance Routing Path (IRP).
10. Container, then, forwards this message to next service depending upon IRP.
11. Finally, the analyzer can change the business process by reconfiguring ARP.

Table 4 demonstrates the routing aspect that had been covered by several authors. Every business organization that deals in ESB products provides its own API called as enterprise integration patterns (EIP). These EIP supports in recognizing message format from other application service domains. Routing is robust feature for ESB that could propagate message to its destination. To federate routing, MOM (message oriented model) is introduced that is based on event-driven architecture (EDA) and SOA [16]. Different Dynamic Reliable Service Routing methodologies have been proposed by different authors like - Dynamic Routing in Enterprise Service Bus (DRESR), Efficient & Reliable Dynamic Service Routing (ERDSR) [23, 24, and 25].

Table 4: Routing, Protocol Transformation and Pattern Recognition in ESB

Reference	Service Routing	Enterprise Pattern	Message Transformation	Comment
Martin Breest (2007)	Discuss message routing facility via MOM and its techniques- itinerary based routing, and content based routing	Promise to construct robust and scalable SOA with MOM	Introduced "Message Oriented Middleware (MOM)" that act as mediation	Brief about service container and discussed how ESB endpoint is similar to servlet in J2EE; service orchestration using BPEL
Xiaoying Bai et al. (2007)	Proposed Dynamic Reconfigurable ESB Service Routing (DRESR)	Includes Instance Routing Path (IRP) with combination of Abstract Routing Table (ART) and Abstract Rout Path (ARP)	Routing strengthen and assured message delivery and help in protocol conversion. Implement D-Mule, open source ESB	Does runtime service testing by dynamic service selection
Bin Wu et.al. (2008)	Improved Dynamic Reliable Service Routing and extended DRESR	Explained 04 routing pattern – Enricher, Static Recipient List (SRL), Message Aggregator (MA), Wire Tap (WT)	Improved Message routing and its conversion with Dynamic WT	Also, discuss important reliable message routing specifications
Sanjay P. Ahuja et. al. (2011)	Work on Content based Routing with scenario to Direct Proxy, Content Based Proxy and Transformation Proxy	Discuss importance of EAI in ESB	Explained Message transformation as core functionality in ESB	Done statistical analysis for calculating throughput or mean response time
Peng Xu et. al. (2013)	Proposed Efficient & Reliable Dynamic Service Routing (ERDSR)	Presents ERDSR Pattern to improve efficiency in ESB message routing	Does message processing or transformation	Made comparison for DRESR, DRSR and ERDSR

4. Summary for Exiting ESBs

It has been surveyed that many organizations, especially working in ESB and SOA domain, provide several facilities related to business application. Though, ESB can be a backbone for any enterprise, so it must be agile enough to provide reliability, flexibility and security for its clients. This is the reason, why organization like Microsoft, Oracle, and IBM are focusing on ESB technology.

Available set of ESBs in the market supports SOA platform for its execution. Every enterprise organization provides set of enterprise integration patterns (EIP) against cross platform support [50]. These patterns help in determining message format and related fields in services. ESB also supports basic fundamental feature like message routing, transformation, protocol conversion and support for dynamic environment. Here, we discuss different ESB product that are available in market.

WSO2 ESB is an open source ESB that provides better compatibility and high performance in terms of service accessibility. This may be the finest option in the available class of ESBs as it supports almost all features of ESB like - Service governance, monitoring and management, virtual environment and so on. According to eBay Inc., for online shopping, it uses WSO2 Enterprise Service Bus for executing and processing transactions, which is more than 1 billion per day. However, it fails to provide proficient security metrics for authentication [39].

Talend* Enterprise ESB is reliable, scalable and secure choice for service bus for any organization. It is non-open source and high performance ESB solution and supports the deployment of services in both on-premises and over cloud. But, it is not lightweight service bus and this is the reason why Talend* Enterprise ESB does not perform well in large-scale operational batch processing [46].

AdroitLogic UltraESB is lightweight service bus, which is capable of supporting different transport and message formats. It is the first ESB, which supports “Zero-Copy proxy” messaging with non-blocking input/output capabilities. It has the ability to dynamically *load or reload* deployment service units. But, it does not support service composition well. It used WS-Security for configuring service to limited extends. UltraESB provides weak compatibility with cloud network [40].

Mule ESB is an open source and lightweight service bus based on Java Messaging Service (JMS) and responds quickly. It easily establishes connection and enables *apps* to exchange messaging data. However, Mule ESB does not provide service governance and security metrics. Mule ESB could be a moderate option for using a service bus for delivering services with medium performance [27].

Apache Synapse ESB is open source, lightweight and high-performance service bus. It also provides deploying services as Apache Synapse server instances. It helps in monitoring and governance of services across network. It also implements complex routing scenario with conditions like non-blocking HTTP/S transports for *fast-mode* of interactions. But, Apache Synapse ESB does not support composition of service components, called *module*. It offers minimum security authentication. It may result in weak federation with cloud network [49].

Red Hat JBoss Fuse is open source ESB, which is based on *Apache ServiceMix* integration platform that supports Java Business Integration (JBI) and Open Service Gateway initiative (OSGi) specification. *Red Hat Inc.* took over *FuseSource* from *Progress Software Corporation* in June 2012. Fuse ESB was renamed as *JBoss Fuse*. It is robust SOA infrastructure that delivers standardized methodology, server and tools to integrate service integration with complex applications. It supports medium performance and limited security metrics [45].

TIBCO's ActiveMatrix® Service Bus is lightweight, non-open source and high performance ESB, which helps in organizing services by routing and transforming data formats and transport protocols. It allows fast development, nearly zero coding & low maintenance. It also manages secure communication between services and application system. It could be good option for commercial organization to adopt this ESB as architectural infrastructure [35].

IBM Websphere is a java centered ESB that supports strong integration for web-based services. It is more competent, reliable, secure, and high performance ESB. It also non-open source, permits high availability and robust failure capability to application servers. IBM facilitates with its own graphical editor, which has *drag and drop* functionality for developing mediation flows. It also helps in reducing costs with fast and flexible application service integration. This could be better option for commercial organization [37].

Window Azure Bus is mainly used for cloud based services architectural platform. It is non-open source and high performance ESB that can also be deployed on-premise. Several services are maintained by Microsoft Corp. through Windows Azure data centers, located across the world. It provides better service governance, monitoring, and

security management policy rules in handling services to facilitate its client. This could also be better option for commercial organization [36].

Sonic ESB connects, mediates and controls services through its own interface environment. It is pure proprietary ESB. It permits fast and secure communications with transactional failover recovery. It also provides reusability among service components in SOA system. It handles deployment service via Sonic Workbench Run Framework. It consists of different API and communications logics that are necessary to debug any errors in service integration [30].

Fiorano ESB is a JMS messaging-based ESB that manage communication interactions between multiple services. It helps in reducing complexity and increase flexibility by re-using service modules. It is a non-open source, reliable and high performance ESB. There are various commercial products by *Fiorano Inc. (USA)* available in market like Fiorano SOA, Fiorano Cloud, and FioranoMQ, Fiorano API Management etc., which extends different features of SOA [9].

Oracle ESB is commercial, flexible and high performance ESB. It allows open standards to establish, transform, and route business data between multiple services. It enables governance and monitoring of business audit log data without influence on current services. It supports service orchestration with flexible integration between multiple services. It provide secure channel for communication via routing messages in ESB with limited functionality [47].

Table5 presents above information into concrete form along with additional data of control and process management of ESB product. Here, Market strategy specifies the current punch line from that particular business organization to attract its customers towards ESB product.

Table 5: Market overview, handling and control management for available set of ESBs

Reference	Market Strategy	Control Management	Process Handling	Comment
Oracle® www.oracle.com	Robust, scalable, flexible, Adapters/Transport Module	Message Tracking, Load Balancing, Message Throttling (No. of Message/time)	Support stateless message flows, secure configuration	Discuss about architectural behavior of ESB
WSO2 ESB www.wso2.com	Fault tolerant mediations, priority-based routing, service chaining	Governance Registry, WS-Security, LDAP, Kerberos, OpenID, SAML, XACML	Flexible logging support, Centralized configuration management	Does File Processing support
TIBCO www.tibco.com	Assimilate with any third party technology, Do more faster and spend less	Business Events deployment, Memory object management	Reduce complexity, enrich visibility, promote reusability	Define ESB as shared superhighway for data, messages, and events
JBoss Fuse www.jboss.org	Pattern based integration framework, fully certified Java™ EE platform,	OpenShift Platform-as-a-Service (PaaS), connectors for JDBC, FTP/SFTP,	Dynamic configuration and management, easily deploy or update services	Provide elastic footprint to integration beyond data centers
Apache ServiceMix servicemix.apache.org	flexible, open-source integration container, powerful runtime platform	OSGi-based server runtime, RESTful web services	BPM engine via <i>Activiti</i> , full JPA support via <i>Apache OpenJPA</i> ,	ESB exclusively powered by OSGi
Talend ESBtalend.com/resource/esb	Powerful, Flexible Open Source ESB, Intelligent routing and mediation	drag-and-drop Eclipse environment, Talend ESB incorporates with Apache ServiceMix	Reduced costs than proprietary products	Avoids foster agility and vendor lock-in
IBM Web Sphere www.ibm.com/wsesb	Dynamic connectivity service solution, business agility	Service Data Objects (SDO) and Service Component Architecture (SCA)	Facilitates different protocols like JMS, SOAP, J2EE Connector Architecture (JCA) adapters	Message Logging and auditing
Fiorano ESBwww.fiorano.com	Real-time solutions addressing, obviating hardcoded dependencies	Build on a MicroServicesbasedarchitecture, agility in authentication, authorization and encryption Support protocol like XACML,	Message-Driven Microservice Model, Supports sophisticated XSLT transformations	Supports dynamically configurable event-pipelines with distributed environment
AdroitLogicUltraESB www.adroitlogic.org	Development, Configurable and Testing for dynamic computing	JTA XA transactions, WebSockets, MLLP/S, Protocol Buffers, JSON, AMQP, SFTP	mediation logic in JVM based on scripting language like JSR 223	provision for Zero-Copy proxy
OpenESB www.open-esb.net	Best balance between scalability, reliability and ergonomic development	Relying on standard JBI, WSDL, BPPEL, XML, SOAP, LDAP, REST, Task Scheduler	Very low TCO (Total Cost of Ownership), lightweight JBI implementation in Java.	Allows integrating with legacy systems. Initially, launched by Sun Microsystems
Petals ESB www.petals.ow2.org	Adaptable to large scale infrastructures, highly distributed topology and modularity	Supports JBI (JSR 208) industry specification, SOAP, BPPEL, SCA, XSLT, XSD, EIP, RMI, CSV transformation	JBI pluggable components, fractal deployment framework	SOA based interconnect heterogeneous systems

Progressive Sonic ESB www.progress.com	Continuously Available Architecture (CAA), most Reliable and Scalable JMS-based	JAX*RS and JAX*WS annotated POJO model, support for SOAP, UDDI and WSDL	Guaranteed message delivery, dynamic routing architecture	Improved Visibility and Diagnostics, manage authentication locally
--	---	---	---	--

4.1 Criteria for comparison

This sub-section describes several criteria for the purpose of comparing existing ESBs in the market. Such categorization is based on whether a particular ESB is open source or not, provides graphical interface to its clients, performance, flexibility and service governance. Existing ESBs could also be compared on basis of security metric, deployment scenario, virtual environment support, and essential feature like message routing, transformation, protocol conversion etc. A brief description for these criteria has been discussed below:

- *Enterprise Integration Pattern* is a standard set of integration ‘templates’ that have been used by software architects and developers in designing and implementing integration based solutions more rapidly and reliably. These patterns have been provided by every IT based company to supports its clients in service identification and accessibility [29, 41].
- *Monitoring and control* in ESB define analyzing situations for service integration and message routing during its execution. ESB controls message instance processing with mediation flow. It also evaluates overall progress of multiple services that support deployment of applications.
- *Deployment* is a process of enabling service applications for testing a business environment. It analyzes whether a particular business logic execute its functionality to deliver better services.
- *Supports for SOA Platform* in ESB specify whether an ESB is extending the features for SOA. It also provides support for multiple services stack [33].
- *Service Governance* in ESB states how uniformly business logic rules are applied in service lifecycle management with SOA systems. Governance also defines policies through which clients can access and integrate with other services. It also incorporates security metric associated to service monitoring.
- *Service composition* defines the integration of service components, called *modules*. It provides the interaction among these modules in delivering services across the network. It also deals with supporting service discovery, configuration, modeling, and validation [8, 38].
- Cloud adapters/connectors in ESB are designed to provide connectivity between applications like Siebel, SAP, Oracle apps etc. It provides an interface for service components that has been utilized in integration process. Adapter/connector never depends on ESB version, nor does it flow with ESB implementation. It just simply plug-in with ESB through well-defined connector/end-point.

Table 6 depicts the association between different criteria and available set of ESB products from different companies like TIBCO Active Matrix, Mule ESB, IBM Websphere, Apache Synapse, Oracle Enterprise Service Bus, Windows Azure Service Bus and many more

Table 6: Comparison criteria for existing ESBs in the market

	TIBCO Active Matrix	Fiorano ESB	Mule ESB	IBM Websphere	Apache Synapse	Red Hat JBoss Fuse ESB	Sonic ESB	WSO2 ESB	Talend Enterprise ESB	AdroitLogic UltraESB	Oracle ESB	Windows Azure Service Bus
Open-source	✗	✗	✓	✗	✓	✓	✗	✓	✗	✓	✗	✗

Graphical Interface	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
Visual Environment	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Supports complete SOA Platform	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Lightweight Service Governance	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓
Service Composition	✓	✓	✗	✓	✓	✓	✗	✓	✓	✗	✓	✓
Performance	High	High	Medium	High	High	Medium	High	High	High	Medium	High	High
Cloud Adapters / Connectors Management & Security Metric	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗	✓	✓
Enterprise Integration Patterns	Moderate	Limited	Limited	High	Limited	Limited	Moderate	Moderate	High	Moderate	High	Moderate
Monitoring & Control	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Deployment	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
Flexibility	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Essential features like routing, message transformation, protocol support and mediation	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

5. Discussion

There is no standard definition for ESB but several authorities have termed ESB as middleware that support multiple services integration and communication with other services. To deliver, ease and fast services through ESB, software service quality and features must be rapid enough to handle demand of business. Open source, agility and scalability are major challenges in setting up the enterprise software needs. To gear up with these challenges, one must be committed to open standards and interoperability to proprietary solutions. Open source software could be better solution to avoid the vendors' features lock-in and temporary availability. It also helps in resolving issue for application integration and support more sophisticated and fast deployable paradigm over different range of environments.

Today, APIs have become actual mode of publicity to get famous of your business. New start-up has launched its API to exposing data and internal business processes to compete the market. Such APIs are primarily exposed using REST protocol with JSON. Now days, REST and JSON together resembles the combination of SOAP and XML [42]. These APIs also help in service identification like service version, its type and format. Usually, APIs are issued in the form of integration pattern that have been provided by its building organization. ESB could best deliver these API and help in minimizing the integration risk. It would atomically improve the Return on Investment (RoI) by adopting standards for SOA systems.

5.1 Opportunity with ESB

ESB is best in-class to integrate, develop, deploy and manage multiple services on common platform. ESB is order to perform and deploy services with SOA technology with followings:

- To moderate costing issues
- To speed-up IT implementations
- To reduce IT complexity

Organizations can align its business with use of appropriate ESB selection in specific domain of services and reuse multiple applications through web services [18]. ESB also enables developers to provide a rich, integrated and interactive development environment to build federated services. This inclusion of ESB improves rapid development of integration logic, which does not require detailed knowledge about J2EE or Java. ESB supports virtualization and handling features that execute and extend the core functionalities of SOA. ESB could be adopted due to following reasons:

- Easy-to-use mediation proficiency
- Extensive connectivity with existing IT assets
- Integrated Framework for development & deployment
- Improve manageability via single point of control
- Better flexibility to its existing systems
- Ability to extend your environment
- Improved performance & optimized networks
- Maximized business ability & adaptations
- Secure and dynamic routing for its messages
- API adaptable to know enterprise pattern
- Implement and execute the virtualization without revealing identity & location

5.2 Threat, Prevention & Gap with ESB

There is no permanent solution to the problems that occurred during integration of services. It is advisable to beware of terms and conditions that made at time of purchase of proprietary ESB [51]. At first, decision must be taken whether particular ESB framework is sufficient and meet the entire requirement of project at organization. It is better to judge the necessity of ESB that which ESB is best suitable and how its complexity could affect overall project growth.

Proprietary and open source ESB both are better option depending upon the requirement. Sometimes, open source ESB performances are far better than commercial/ proprietary ESB [29]. At some junctions, these both ESBs have also similar characteristics. Thus, it is suggested at the beginning of project, check the necessities and demand in selecting ESB. Which would be better solution - proprietary or open source software? There could be following threats and questions that must be kept in mind by developer or architecture designer for choosing ESB for its business:

- **Manageability:** Point out the overall handling and monitoring support with ESB. Questions like GUI support for monitoring services? How do you control the product?
- **Developer Support & Guidance:** On using particular ESB, How much technical & social support provided from its developer firm? How many tutorials, articles, and videos available? Active list or public forums available? Support would be online or offline, onsite or offsite, preferred communication language?
- **Costing:** This is very important aspect for choosing ESB product. This may include questionnaire - What is total cost of ownership (TCO) of particular ESB product? What does its maintenance cost? What is its subsidiary products requirement costing? What are rental charges for its cloud adapter, if required?
- **SLA Terms & Conditions:** Point out necessary condition and term for the agreement made at time of purchase of ESB. What would be guaranteed period for terms? What governing policy bears? Which part of services could be on rental basis?
- **Usability:** suggest on questions like - Which development environment support? How installation is carried out? How many supporting tools are required in its installation?
- **Technical Requirement:** Meeting all demands and requirements govern by project? Offered functionality durability?
- **Flexibility:** Brings questionnaire related to customization of ESB product. How functionalities for product could be set according to own necessities?
- **Extensiveness:** Suggest on possibilities of extension of product. This may include question like - Is ESB product expands to external environment? Such interface based on standards or not?

- *Adapter/Connectors*: Check the availability, whether developing firm of ESB provide connector/adaptor for other technologies. How customization for own adapter can build? Would these adapters compatible for B2B products? Are these adapters fulfilling all the technologies demands?
- *Licensing*: Shows the up-gradation and transparency measures provided from developer firm of ESB Product. This may include – Which kind of license methodology is used? Is ESB product is upgradable for free or not? Is change in requirement, may change overall license plan? Is a downgrade possible?

The Gap with ESB discusses the issues that have been found with extensive working beyond ESB. The specific business applications that are complex, dynamic and highly computational, requires lot of effort in processing and somewhat more implementation than ESB. This introduces a new concept called - '*Integration Suite*'. This kind of Integration methodology not only offers all feature of ESB, but also includes the functionality like Business Activity Monitoring, Master Data Management, and Business Process Management (BPM). There could be more area that explores architectural benefit, which starts with "*Microservices Architecture*". This architecture talks about distributed and flexibility in delivering services. ESB sometimes work like central hub, but with distributed agent. Many vendors are not interested in selling ESBs as it has central integration middleware architecture [48].

6. Conclusion

ESB is typical infrastructure in software engineering area. This paper concluded with duties of ESB. It provides the support for services, security handling, and monitoring the quality of service. It has been analyzed that concept of ESB is to connect various service components together i.e. service containers. It is good option for building platform in enterprise integration, as it protects and hide transport protocols information. It has capability to transport messages or data more securely. It also used to integrate several services from hybrid and homogeneous systems as common solution for middleware. ESB also improves the architectural benefit associated with enterprise solution for service integration and well organized service handling. By adopting ESB as their business solutions, one can reduce the risk that involved the data management issues and resultant would automatically increase the RoI.

References

- [1] Jianqiang Hu, FengELuo, JunLi, Xin Tong, Guiping Liao. SOA-based Enterprise Service Bus. In: IEEE International Symposium on Electronic Commerce and Security (ISECS 2008); 2008. p. 536-539.
- [2] Gang Li, Jian Xiao, Chun Li, Sen Li, Jingde Cheng. A Comparative Study between Soft System Bus and Enterprise Service Bus. In: IEEE International Conference on Computer Science and Service System (CSSS 2012), 2012. p. 557-561.
- [3] UenoK, Tatsubori M. Early Capacity Testing of an Enterprise Service Bus. In: IEEE International Conference on Web Services (ICWS), 2006, p. 709-716.
- [4] Schmidt MT. The Enterprise Service Bus: Making service-oriented architecture real. IBM Systems Journal, 2005; 44(4): 781-797.
- [5] Cheng J. Persistent Computing Systems as Continuously Available, Reliable, and Secure Systems. In IEEE First International Conference on Availability, Reliability and Security (ARES), 2006. p. 1-8.
- [6] Psiuk M, Bujok T, ZielinskiK. Enterprise Service Bus Monitoring Framework for SOA Systems. IEEE Transaction on Services Computing, 2012; 5(3): 450-466.
- [7] Riad AM, Hussan AE, Hassan QF. Design of SOA-based Grid Computing with Enterprise Service Bus. International Journal on Advances in Information Sciences and Service Sciences, 2010; 2(1): 71-82.
- [8] Ning, F, Junhua, D, Yan G. A Service Composition Environment based on Enterprise Service Bus. In: IEEE 11th Intl Conf on Ubiquitous Intelligence & Computing and IEEE 11th Intl Conf on Autonomic & Trusted Computing and IEEE 14th Intl Conf on Scalable Computing and Communications and Its Associated Workshops and IEEE 11th Intl Conf on Ubiquitous Intelligence & Computing and 2014 IEEE 11th Intl Conf on Autonomic & Trusted Computing and 2014 IEEE 14th Intl Conf on Scalable Computing and Communications and Associated Symposia/Workshops, 2014, p. 738-743.
- [9] Fiorano ESB. <http://www.fiorano.com/> [Viewed on 21.08.15]
- [10] Liu Y, Gorton I, Zhu L. Performance Prediction of Service-Oriented Architecture based on an Enterprise Service bus. In: 31st Annual International Computer Software and Application Conference, Vol. 1; 2007, p. 327-334.
- [11] Bidgoli AM, Nabhani P. Intelligent Conceptual Message Routing in Enterprise Service Bus (ESB). In: World Congress in Computer Science, Computer Engineering and Applied Computing, 2011.
- [12] Wu J, Tao X. Research of Enterprise Application Integration Based on ESB. In: 2nd International Conference on advanced Computer Control (ICACC), 2010.
- [13] Lewis GA, Smith DB, Kontogiannis K. A Research Agenda for Service-Oriented Architecture (SOA): Maintenance and Evolution of Service-Oriented Systems. Software Engineering Institute, Carnegie Mellon University, 2010.
- [14] Zielinski K, Szydło T, Szymacha R, Kosiński J, Kosińska J, Jarzab M. Adaptive SOA Solution Stack. IEEE Transaction on Services Computing, 2012; 5(2), 149-163.

- [15] Zmuda D, Psiuk M, Zieliński K. (2010). Dynamic Monitoring Framework for the SOA Execution Environment. *Procedia Computer Science*, 2010; 1(1): 125-133.
- [16] Breest M. An Introduction to the Enterprise Service Bus. 2006
http://www.cin.ufpe.br/~in1062/intranet/bibliografia/fose-the_enterprise_service_bus.pdf
- [17] Yin J, Chen H, Deng S, Wu Z. A Dependable ESB Framework for Service Integration. *IEEE Internet Computing*, 2009; 13(2), 26-34.
- [18] Sun CA, El Khoury E, Aiello M. Transaction Management in Service-Oriented Systems: Requirements and a Proposal” *IEEE Transaction on Services Computing*, 2011; 4(2): 167-180.
- [19] Ahuja SP, Patel A. Enterprise Service Bus: A Performance Evaluation. *Journal of Communications and Network*, 2011, 3(3): 133-140.
- [20] Chen M, Chi VPJ, Li HC. An Enterprise Architecture Approach to Building a Service-Oriented Enterprise. In: 6th International Conference on Service Systems and Service Management (ICSSSM), 2009. p.704-709.
- [21] Ortiz Jr. S. Getting on Board the Enterprise Service Bus. *IEEE Computer Magazine*, 2007; 40(4): 15-17.
- [22] Aberdeen Group (USA). Enterprise Service Bus: An SOA Middleware Foundation. 2006.
www.fiorano.com/docs/aberddeen.pdf
- [23] Xu P, Du W. ERDSR: Efficient and Reliable Dynamic Service Routing in Enterprise Service Bus. In: 3rd IEEE International Conference on Intelligent System Design and Engineering Applications (ISDEA), 2013. p. 712 – 715.
- [24] Wu B., Liu S, Wu L. Dynamic Reliable Service Routing in Enterprise Service Bus. In: IEEE Asia-Pacific Services Computing Conference, 2008. p. 349–354.
- [25] Bai X, Xie J, Chen B, Xiao S. DRESR: Dynamic Routing in Enterprise Service Bus. In: IEEE International conference on e-Business Engineering (ICEBE), 2007. p. 528–531.
- [26] Bo D, Kun D, Xiaoyi Z. A High Performance Enterprise Service Bus Platform for Complex Event Processing. In: 7th IEEE International Conference on Grid and Cooperative Computing, 2008. p. 577–582.
- [27] Brebner P. Service-Oriented Performance Modeling the MULE Enterprise Service Bus (ESB) Loan Broker Application. In: 35th IEEE Euro micro Conference on Software Engineering and Advanced Applications, 2009. p. 404-411.
- [28] Ruzhi X, Jin B, Yufei W. The Research and Implementation of Power Application System Integration Based on Enterprise Service Bus. In: IEEE International Conference on Intelligent Computing and Intelligent Systems (ICIS), 2010. p. 466 – 469.
- [29] Huang W, Sun BC, Zhao H, Hu YS. Enterprise Service Bus Based on OSGi. In: 22nd International Conference on Industrial Engineering and Engineering Management 2016, pp. 233-245.
- [30] Sonic: Enterprise Service Bus. <http://www.aurea.com/technology-solutions/enterprise-service-bus>, 2013. [Viewed on 21.08.15]
- [31] Ji-chen J, Ming G. Enterprise Service Bus and an Open Source Implementation. In: IEEE International Conference on Management Science and Engineering (ICMSE), 2006, p. 926 – 930.
- [32] IBM Research Work. Understand Enterprise Service Bus scenarios and solutions in Service-Oriented Architecture. <https://www.ibm.com/developerworks/library/ws-esbscen/> [Viewed on 21.08.15]
- [33] IBM, “Exploring the Enterprise Service Bus: Discover how an ESB can help you meet the requirements for your SOA solution”, URL: <https://www.ibm.com/developerworks/library/ar-esbpat1/>
- [34] Lin KJ, Panahi M, Zhang Y, Zhang J, and Chang SH. Building Accountability Middleware to Support Dependable SOA. *IEEE Internet Computing*, 13(2); 2009: 16-25.
- [35] TIBCO ActiveMatrix® Service Bus. <http://www.tibco.co.in/> [Viewed on 21.08.15]
- [36] Window Azure Bus. <http://azure.microsoft.com/en-us/services/service-bus/> [Viewed on 21.08.15]
- [37] IBM Research Work. Exploring the Enterprise Service Bus: Part 4: Federated connectivity in the enterprise. http://www.ibm.com/developerworks/websphere/library/techarticles/0901_flurry/0901_flurry.html
- [38] Tang X, Sun S, Yuan X, Chen D. Automated Web Service Composition System on Enterprise Service Bus. In: 3rd IEEE International Conference on Secure Software Integration and Reliability Improvement, 2009. p. 9-13.
- [39] WSO2 ESB. <http://www.wso2.com/products/enterprise-service-bus/> [Viewed on 21.08.15]
- [40] Adroit Logic Ultra ESB. <http://www.adroitlogic.org/products/ultraesb.html> [Viewed on 21.08.15]
- [41] Integration Pattern Overview. <http://www.eaipatterns.com/eaipatterns.html> [Viewed on 21.08.15]
- [42] Harcuba O, VrbaP. Unified REST API for supporting the semantic integration in the ESB-based architecture. In: IEEE International Conference on Industrial Technology (ICIT), 2015. p.3000-3005.
- [43] Benosman R, Barkaoui K, Albriex Y. Design and implementation of a massively parallel ESB. In: 11th International Symposium on Programming and Systems (ISPS), 2013, p. 89-95.
- [44] ShiK, GaoF, XuQ, Xu G. Integration framework with semantic aspect of heterogeneous system based on ontology and ESB. In: 26th Chinese Control and Decision Conference (2014 CCDC), 2014. p.4143-4148.
- [45] Red Hat JBoss ESB. <http://www.jbossesb.jboss.org/> [Viewed on 21.08.15]
- [46] Talend Enterprise ESB. <http://www.talend.com/products/esb/> [Viewed on 21.08.15]
- [47] Oracle ESB. <http://www.oracle.com/technetwork/middleware/service-bus/overview/index.html> [Viewed on 21.08.15]
- [48] Fowler M. Micro Services Architecture - Death of Enterprise Service Bus (ESB). Thought Work Incorporation, USA <http://www.kai-waehner.de/blog/2015/01/08/micro-services-architecture-death-enterprise-service-bus-esb/> [Viewed on 21.08.15]
- [49] Apache Synapse ESB. Retrieved From: <https://synapse.apache.org/> [Viewed on 21.08.15]

- [50] Ferreira A, Pereira A, Rodrigues N, Barbosa J, Leitao P. Integration of an agent-based strategic planner in an enterprise service bus ecosystem. In 13th IEEE International Conference on Proceedings of Industrial Informatics (INDIN), 2015, p. 1336-1341.
- [51] RajKumar N, Vinod V. (2015). Integrated Educational Information Systems for Disabled Schools via a Service Bus using SOA. *Indian Journal of Science and Technology*, 2015; 8(13): p. 1-7.
- [52] Kumari S, Rath SK. Performance comparison of SOAP and REST based Web Services for Enterprise Application Integration. In: *IEEE International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, 2015, p. 1656-1660.
- [53] González, L., Laborde, J. L., Galnares, M., Fenoglio, M., &Ruggia, R. An adaptive enterprise service bus infrastructure for service based systems. In: *Springer Service-Oriented Computing (ICSOC) Workshops*, 2014, p. 480-491.
- [54] Pan G, ZhangL, Wu Z, Li S, Yang L, Lin M, Shi Y. Pervasive Service Bus: Smart SOA Infrastructure for Ambient Intelligence. *IEEE Intelligent Systems*, 2014; 29(4), 52-60.
- [55] Ulltveit-Moe N, Oleshchuk V. A novel policy-driven reversible anonymisation scheme for XML-based services. *Elsevier Information System*, 2015; 48(2015): 164-178.
- [56] Alam K A, Ahmad R, Akhunzada A, Md Nasira MHN, Khan SU. Impact analysis and change propagation in service-oriented enterprises: A systematic review. *Elsevier Information System*, 2015; 54(2015): 43-73.

Highlights

- A detailed and empirical survey for Enterprise Service Bus (ESB) in Service-Oriented Architecture (SOA) system.
- Presents core functionality of ESB like message routing, message handling and service stack.
- Concept of ESB in information/message handling from multiple services.
- Software Industry assessment in adoption of ESB to facilitate their service accessibility architecture.